

SNA4DS cheatsheet

Automatically generated from the current snafun vignette tables and figures.

Contents

- [Main packages in the SNA4DS course](#)
 - [Overview of igraph and network graph objects](#)
 - [The snafun package](#)
 - [Creating a graph object](#)
 - [Additional graph creation functions](#)
 - [Converting between graph classes](#)
 - [Manipulating the graph object](#)
 - [Graph level indices](#)
 - [Components and communities](#)
 - [Vertex-level indices](#)
 - [Dyad-level indices](#)
- [Plotting](#)
 - [Plotting in snafun](#)
 - [Statistical models](#)
 - [Overview table](#)
 - [Network autocorrelation models](#)
 - [Conditional Uniform graphs \(CUG\)](#)
 - [QAP test](#)
 - [QAP linear regression](#)
 - [Temporal networks](#)
 - [Network generation and manipulation](#)
 - [Network measures and descriptives](#)
- [QAP logistic regression](#)
 - [Terms classification for every Exponential Random Graph Model \(ERGM\)](#)
 - [Bipartite ERGMs](#)
 - [ERGM goodness of fit and effect interpretation](#)
- [ERGM for temporal networks](#)
 - [Participation shifts](#)

In this *cheatsheet* we summarize the main R functions that are used in the SNA4DS course. We will not explain the underlying concepts here, but refer you to the lectures, labs, and slides of the course for that.

The aim of this cheatsheet is that it provides you with an overview of the main functions you will need throughout the course. We hope that it can provide a useful reference for you, as you develop and apply your network analysis skills.

NOTE:

Most functions have multiple arguments. Our aim is in this cheatsheet not to show and discuss the various arguments that exist, because that would yield an unwieldy and very long document. Rather, we recommend you use your R skills and use the help function `?` and `help` and other approaches we teach you in this course to learn about the details of a specific function. If you still can't figure it out, contact us and we'll assist you.

Main packages in the SNA4DS course

Overview of igraph and network graph objects

There are two main packages for basic graph generation and manipulation: the `igraph` package and the `statnet` package. Actually, `statnet` is a suite of packages that work together. In this course, we will make use of several packages from the `statnet` suite.

The `igraph` package creates a graph object of type `igraph`. The `statnet` suite creates a graph object of type `network`. Both can generate graphs and do basic manipulation. The `igraph` package provides more mathematical graph functions, while the `statnet` suite provides many statistical models.

The snafun package

The `igraph` package and `statnet` suite are jointly very powerful, but they each require graph objects with specific structures. If you want to combine functions from `igraph`, `network`, and `sna`, you often end up converting between `igraph`, `network`, `matrix`, and `edgelist` representations.

Believe it or not, this is a pain and quite annoying.

THE snafun PACKAGE TO THE RESCUE!

- It provides a fairly consistent API, so you don't have to constantly re-learn different argument conventions.
- Most functions work on `igraph`, `network`, matrices, and edgelists/data frames.
- It lets you focus on the fun of network analysis rather than the friction of conversions and mismatched interfaces.

Creating a graph object

Graph creation	
CREATE AN EMPTY NETWORK	
snafun	snafun::create_empty_graph(n_vertices, directed = TRUE, graph = c("igraph", "network"))
igraph	igraph::make_empty_graph()
network	network::network.initialize(n)
CREATE A MANUAL (LITERAL) NETWORK	
snafun	snafun::create_manual_graph(A -+ B -+ C, simplify = TRUE, graph = 'igraph')
igraph	igraph::graph_from_literal(A -+ B -+ C, simplify = TRUE)
network	-
CREATE A RING NETWORK	
snafun	-
igraph	igraph::make_ring()
network	-
CREATE A STAR NETWORK	
snafun	-
igraph	igraph::make_star()
network	-
CREATE A RANDOM GRAPH WITH GIVEN DENSITY	
snafun	snafun::create_random_graph(n_vertices = 10, strategy = "gnm", # to fix the number of edges m = 27, # number of edges required, resulting density = .3 directed = TRUE, # or FALSE graph = c("igraph", "network")) # pick one snafun::create_random_graph(n_vertices = 10,

¹ The output is a matrix object, not a 'network' object.

² This function is quite unintuitive and cumbersome

Graph creation		
		<pre>strategy = "gnp", # to fix the probability edges p = .3, # probability for each edge, yields approx. density of .3 directed = TRUE, # or FALSE graph = c("igraph", "network"))</pre>
igraph		<pre>igraph::sample_gnm() igraph::sample_gnp() igraph::make_directed_graph(n = 10, edges = 27) igraph::make_undirected_graph(n = 10, edges = 27)</pre>
network ¹		<pre># 10 vertices, density on average .3 sna::rgraph(n = 10, m = 1, tprob = 0.30, mode = 'digraph') # 10 vertices, 27 edges (ie. density = .3) sna::rgnm(1, 10, m = 27)</pre>

CREATE A RANDOM GRAPH WITH GIVEN DYAD CENSUS

snafun		<pre>create_census_graph(n_vertices = 10, mut = 8, asym = 25, null = 12, method = 'exact', graph = 'igraph')</pre>
igraph	–	
network ¹		<pre># 5 networks, each with 10 vertices, and 8 M, 25 A, and 12 N dyads sna::rguman(n = 5, nv = 10, mut = 8, asym = 25, null = 12, method = "exact")</pre>

CREATE A RANDOM GRAPH WITH GIVEN COMMUNITY STRUTURE

snafun		<pre>snafun::create_community_graph(communitySizes = c(10, 20, 30), p_intra = c(0.3, 0.2, 0.3), p_inter = 0.2, p_del = 0, graph = 'igraph')</pre>
igraph	–	
network	–	

CREATE A GRAPH WITH GIVEN COMPONENTS

snafun		<pre>snafun::create_components_graph(</pre>
--------	--	---

¹ The output is a matrix object, not a 'network' object.

² This function is quite unintuitive and cumbersome

Graph creation

	<pre>n_vertices, directed = FALSE, membership = NULL, graph = 'igraph')</pre>
igraph	-
network	-

CREATE RANDOM BIPARTITE GRAPH

snafun	<pre>snafun::create_bipartite(n_type1, # number of vertices of type 1 n_type2, # number of vertices of type 2 strategy = c("gnp", "gnm"), p, # probability of each cross-type edge m, # number of cross-type edges directed = FALSE, mode = c("out", "in", "all"), graph = c("igraph", "network"))</pre>
igraph	<pre>igraph::sample_bipartite_gnp(n1, n2, p)</pre>
network ²	<pre>network::network.bipartite()</pre>

CREATE GRAPH OBJECT FROM INPUT DATA

snafun	<pre># `x` can be: # - an edgelist (in data.frame format) # - an incidence matrix (in matrix format)--for a bipartite graph # - an adjacency matrix (in matrix format) # - `vertices` can be a data.frame containing vertex attributes snafun::to_igraph(x, bipartite = FALSE, vertices = NULL) snafun::to_network(x, bipartite = FALSE, vertices = NULL)</pre>
igraph	<pre># from an adjacency matrix igraph::graph_from_adjacency_matrix(adjmatrix, mode = c("directed", "undirected", "max", "min", "upper", "lower", "plus"), weighted = NULL, diag = TRUE, add.colnames = NULL, add.rownames = NA) igraph::make_graph(edges, ..., n = max(edges), isolates = NULL, directed = TRUE, dir = directed, simplify = TRUE)</pre>

¹ The output is a matrix object, not a 'network' object.

² This function is quite unintuitive and cumbersome

Graph creation

```
# from an adjacency list
igraph::graph_from_adj_list(adjlist, mode = c("out", "in", "all", "total"),
  duplicate = TRUE)

# from an edgelist
igraph::graph_from_edgelist(el, directed = TRUE)

# from a data.frame
igraph::graph_from_data_frame(d, directed = TRUE, vertices = NULL)

# from a bipartite incidence / biadjacency matrix (in matrix format)
igraph::graph_from_biadjacency_matrix(incidence, directed = FALSE,
  mode = c("all", "out", "in", "total"), multiple = FALSE,
  weighted = NULL, add.names = NULL)

# x can be an adjacency matrix (as a matrix), an incidence matrix (as a matrix),
# or an edgelist (as a data.frame)
network::network(x, vertex.attr = NULL, vertex.attrnames = NULL, directed = TRUE,
  hyper = FALSE, loops = FALSE, multiple = FALSE, bipartite = FALSE, ...)

# if x is a data.frame
network::as.network(x, directed = TRUE, vertices = NULL, hyper = FALSE,
  loops = FALSE, multiple = FALSE, bipartite = FALSE,
  bipartite_col = "is_actor", ...)

# if x is a matrix
network::as.network(x, matrix.type = NULL, directed = TRUE, hyper = FALSE,
  loops = FALSE, multiple = FALSE, bipartite = FALSE, ignore.eval = TRUE,
  names.eval = NULL, na.rm = FALSE, edge.check = FALSE, ...)
```

network

READ DATA FROM A UCINET FILE

```
snafun::read_ucinet(file,
  graph = c("igraph", "network", "matrix", "edgelist"),
  directed = c("auto", "directed", "undirected"))
```

igraph	-
network	-

BUILD AN EDGELIST / NODELIST FROM RAW DATA

```
snafun::make_edgelist(names = NULL, attribute = NULL)
snafun::make_nodelist(names = NULL, attribute = NULL)
```

igraph	-
--------	---

¹ The output is a matrix object, not a 'network' object.
² This function is quite unintuitive and cumbersome

Graph creation

network –

- 1 The output is a matrix object, not a 'network' object.
- 2 This function is quite unintuitive and cumbersome

Additional graph creation functions

After creating a graph, you can also extend it with helper functions such as `snafun::add_vertices()` and `snafun::add_edges()` .

Converting between graph classes

Graph object conversion	
CONVERT TO AN ADJACENCY MATRIX	
igraph	igraph::as_adjacency_matrix(g, sparse = FALSE)
network	network::as.sociomatrix(g) network::as.matrix.network(g)
CONVERT TO AN ADJACENCY LIST	
igraph	igraph::as_adj_list(g)
network	—
CONVERT TO AN EDGE LIST	
igraph	igraph::as_edgelist(g) igraph::as_data_frame(g)
network	network::as.data.frame.network(g) network::as.edgelist(g) # if you want to include edge weights sna::as.edgelist.sna(g)
MAKE THE NETWORK DIRECTED	
snafun	—
igraph	igraph::as_directed(g)
network	—
MAKE THE NETWORK UNDIRECTED	
snafun	# from an adjacency matrix as input snafun::to_symmetric_matrix(g, rule = c("weak", "mutual", "out", "in", "average", "max", "min"), na.rm = TRUE)
igraph	igraph::as_undirected(g) # for example

Graph object conversion	
	<code>igraph::as_undirected(g, mode = 'collapse', edge.attr.comb = list(weight = 'sum'))</code>
network	<pre># from an adjacency matrix as input sna::symmetrize(g) # for example sna::symmetrize(g, rule = "strong")</pre>

PROJECT A BIPARTITE GRAPH

snafun	<pre>snafun::make_projection(x, which = "both") # or request a single partition snafun::make_projection(x, which = "row") snafun::make_projection(x, which = "column")</pre>
igraph	<code>igraph::bipartite_projection(g)</code>
network	—

CONVERT TO A LINE GRAPH

snafun	—
igraph	<code>igraph::make_line_graph(g)</code>
network	—

DICHOTOMIZE A MATRIX (MAKE BINARY)

snafun	<pre># values below 'min' become 0, the rest become 1 snafun::to_binary_matrix(mat, min)</pre>
igraph	—
network	—

Additional graph creation functions

The cheatsheet below focuses on the most common creation routes. The package also contains helper functions for adding vertices and edges after creation.

Convert between various formats Using the consistent functions of snafun				
		OUTPUT		
INPUT	edgelist	matrix	igraph	network
edgelist	to_edgelist()	to_matrix()	to_igraph()	to_network()
matrix	to_edgelist()	to_matrix()	to_igraph()	to_network()
igraph	to_edgelist()	to_matrix()	to_igraph()	to_network()
network	to_edgelist()	to_matrix()	to_igraph()	to_network()
Conversion includes bipartite graphs. Check the arguments for the various options.				

Manipulating the graph object

Manipulate the graph object

PRINT THE GRAPH OBJECT

snafun	snafun::print(x)
igraph ¹	igraph::print.igraph(x)
network ¹	network::print.network(x)

CHECK CHARACTERISTICS OF THE GRAPH OBJECT

snafun	snafun::has_edge_attributes(x)
	snafun::has_vertex_attributes(x)
	snafun::has_vertex_attribute(x, attrname)
	snafun::has_edge_attribute(x, attrname)
	snafun::has_vertexnames(x)
	snafun::has_loops(x)
	snafun::is_bipartite(x)
	snafun::is_connected(x, rule = c("weak", "strong"))
	snafun::is_directed(x)
	snafun::is_network(x)
igraph	snafun::is_igraph(x)
	snafun::is_signed(x)
	snafun::is_weighted(x)
	# various functions, including
	igraph::any_loop(g)
	igraph::any_multiple(g)
	igraph::is_bipartite(graph)
	igraph::is_connected(graph, mode = c("weak", "strong"))
	igraph::is_directed(graph)
	igraph::is_igraph(graph)

¹ Often, but certainly not always, it will also work if you just use `print(x)`

² These functions have many additional arguments

Manipulate the graph object

	<code>igraph::is_named(graph)</code> <code>igraph::is_simple(graph)</code> <code>igraph::is_weighted(graph)</code>
network	<code># various functions, including:</code> <code>network::has.loops(x)</code> <code>network::is.bipartite(x)</code> <code>network::is.directed(x)</code> <code>network::is.multiplex(x)</code> <code>network::is.network(x)</code> <code>sna::is.connected(g, connected = "strong")</code>

ACCESS THE VERTICES AND EDGES

snafun	—
igraph	<code>igraph::V(x)</code> # vertices <code>igraph::E(x)</code> # edges
network	—

EXTRACT VERTEX / EDGE / GRAPH ATTRIBUTES

snafun	<code>snafun::extract_vertex_attribute(x, name)</code> <code>snafun::extract_vertex_names(x)</code> # specific to access the vertex names attribute <code>snafun::extract_edge_attribute(x, name)</code> <code>snafun::extract_graph_attribute(x, name)</code>
igraph	<code>igraph::vertex_attr(graph, name, index = V(graph))</code> <code>igraph::V(graph)\$attributename</code> <code>igraph::edge_attr(graph, name, index = E(graph))</code> <code>igraph::E(graph)\$attributename</code> <code>igraph::graph_attr(graph, name)</code>
network ²	<code>network::get.vertex.attribute(x, attrname)</code> <code>network::get.edge.attribute(x, attrname)</code> <code>network::get.network.attribute(x, attrname)</code>
igraph	—
network	—

¹ Often, but certainly not always, it will also work if you just use `print(x)`

² These functions have many additional arguments

Manipulate the graph object

EXTRACT THE ID OF AN EDGE

snafun	snafun::extract_edge_id(object, ego, alter, edgelist)
--------	---

EXTRACT VERTEX NAMES

snafun	snafun::extract_vertex_names(x)
--------	---------------------------------

igraph	igraph::V(x)\$name igraph::vertex_attr(x, name = "name")
--------	---

network	network::get.vertex.attribute(x, "vertex.names") network::network.vertex.names(x)
---------	--

SET VERTEX NAMES

snafun	snafun::add_vertex_names(x, value)
--------	------------------------------------

igraph	igraph::V(x)\$name <- value igraph::set_vertex_attr(x, name = "name", value = value)
--------	--

network	network::set.vertex.attribute(x, "vertex.names", value) network::network.vertex.names(x) <- value
---------	---

EXTRACT INTERNAL VERTEX IDS

snafun	snafun::extract_vertex_ids(x)
--------	-------------------------------

igraph	—
--------	---

network	—
---------	---

MAKE A MATRIX FROM A VERTEX ATTRIBUTE

snafun	# e.g. to build covariate matrices for ERGM edgecov()/absdiff() snafun::make_matrix_from_vertex_attribute(x, name, measure = c("absdiff", "diff", "sum", "max", "min", "mean", "sender", "receiver", "equal"))
--------	---

igraph	—
--------	---

¹ Often, but certainly not always, it will also work if you just use `print(x)`
² These functions have many additional arguments

Manipulate the graph object

network -

LIST VERTEX / EDGE / GRAPH ATTRIBUTES

snafun
snafun::list_vertex_attributes(x)
snafun::list_edge_attributes(x)
snafun::list_graph_attributes(x)

igraph
igraph::vertex_attr_names(graph)
igraph::edge_attr_names(graph)
igraph::graph_attr_names(graph)

network
network::list.vertex.attributes(x)
network::list.edge.attributes(x)
network::list.network.attributes(x)

EXTRACT ALL VERTEX ATTRIBUTES INTO A DATA.FRAME

snafun
snafun::extract_all_vertex_attributes(g)

igraph -

network -

ADD VERTEX / EDGE / GRAPH ATTRIBUTES

snafun
very flexible and powerful functions to add attributes
in several ways, with the same function
snafun::add_edge_attributes(object, attr_name, value, edgelist, overwrite = FALSE)

snafun::add_vertex_attributes(x, attr_name = NULL, value)
snafun::add_graph_attribute(x, attr_name = NULL, value)

igraph
igraph::set_edge_attr(g, "name", value)
igraph::E(g)\$name <- value

igraph::set_vertex_attr(g, "name", value)
igraph::V(g)\$name <- value

igraph::set_graph_attr(g, "name", value)
g\$name <- value

¹ Often, but certainly not always, it will also work if you just use `print(x)`

² These functions have many additional arguments

Manipulate the graph object

network	<code>network::set.edge.attribute(g, "new_name", value, e = seq_along(g\$mel))</code>
	<code>network::set.edge.value(g, "new_name", value, e = seq_along(g\$mel))</code>
	<code>network::set.network.attribute(g, "new_name", value)</code>
	<code>network::set.vertex.attribute(g, "new_name", value, v = seq_len(network::network.size(g)))</code>

REMOVE VERTEX / EDGE / GRAPH ATTRIBUTES

snafun	<code>snafun::remove_edge_attribute(x, attr_name)</code>
	<code>snafun::remove_edge_weight(x) # remove edge weights</code>
	<code>snafun::remove_vertex_attribute(x, attr_name)</code>
	<code>snafun::remove_vertex_names(x) # specific for the vertex name attribute</code>
	<code>snafun::remove_graph_attribute(x, attr_name)</code>
igraph	<code>igraph::delete_edge_attr(graph, name)</code>
	<code>igraph::delete_vertex_attr(graph, name)</code>
	<code>igraph::delete_graph_attr(graph, name)</code>
network	<code>network::delete.edge.attribute(x, attrname, ...)</code>
	<code>network::delete.vertex.attribute(x, attrname, ...)</code>
	<code>network::delete.network.attribute(x, attrname, ...)</code>

ADD VERTICES OR EDGES

snafun	<code>snafun::add_vertices(x, vertices)</code>
	<code>snafun::add_edges(x, edges)</code>
igraph	<code>igraph::add_vertices(graph, nv, ..., attr = list())</code>
	<code>igraph::add_edges(graph, edges, ..., attr = list())</code>
network	<code>network::add.vertices(x, nv, vattr = NULL, last.mode = TRUE, ...)</code>
	<code>network::add.edge(x, tail, head, names.eval = NULL, vals.eval = NULL, edge.check = FALSE, ...)</code>
	<code>network::add.edges(x, tail, head, names.eval = NULL, vals.eval = NULL, ...)</code>

REMOVE VERTICES OR EDGES

snafun	<code>snafun::remove_vertices(x, vertices)</code>
--------	---

¹ Often, but certainly not always, it will also work if you just use `print(x)`

² These functions have many additional arguments

Manipulate the graph object

igraph	igraph::delete_vertices(graph, v)
	igraph::delete_edges(graph, edges)
network	network::delete.edges(x, eid)
	network::delete.vertices(x, vid)

CONTRACT VERTICES INTO ONE

snafun	snafun::contract_vertices(g, vertices, method = c("min", "max", "union", "add"), attr = NULL, new_name = "set", out_as_igraph = TRUE)	
igraph	—	
network	—	

EXTRACT A SUBSET FROM THE GRAPH

snafun	# single function to do it all snafun::extract_subgraph(x, v_to_keep, e_to_keep)	
igraph	# subset based on vertices igraph::induced_subgraph(g, vids = theVerticesYouWantToKeep)	
	# subset based on edges igraph::subgraph_from_edges(g, eids = theEdgesYouWantToKeep)	
network	# subset based on vertices network::get.inducedSubgraph(g, v = theVerticesYouWantToKeep)	
	# subset based on edges network::get.inducedSubgraph(g, eid = theEdgesYouWantToKeep)	

EXTRACT EGONETS FROM THE GRAPH

snafun	snafun::extract_egonet(x, vertices = NULL, order = 1, type = c("all", "out", "in"))	
igraph	igraph::make_ego_graph(g, order = 1, nodes = V(graph), mode = c("all", "out", "in"), mindist = 0)	
network	sna::ego.extract(dat, ego = NULL, neighborhood = c("combined", "in", "out"))	

¹ Often, but certainly not always, it will also work if you just use `print(x)`

² These functions have many additional arguments

Manipulate the graph object

CLEAN UP THE GRAPH

snafun	<pre>snafun::extract_isolates(x, names = TRUE, loops = FALSE) snafun::remove_isolates(x, loops = FALSE) snafun::remove_loops(x) snafun::extract_loops(x) # which edges are loops snafun::extract_loops_vertex(x) # which vertices carry a loop snafun::has_multiple_edges(x) snafun::extract_multiple_edges(x) snafun::remove_multiple_edges(x)</pre>
igraph	<pre>igraph::simplify(g) # for example igraph::simplify(g, remove.multiple = TRUE, remove.loops = TRUE, edge.attr.comb = list(weight = 'max'))</pre>
network	<pre>sna::isolates(g, diag=FALSE)</pre>

UNION OF GRAPHS

snafun	<pre>snafun::make_union(net1, net2, net3, byname = 'auto')</pre>
igraph	<pre>igraph::union(net1, net2, net3)</pre>
network	—

INTERSECTION OF GRAPHS

snafun	<pre>snafun::extract_intersection(net, net2, net3, byname = 'auto', keep.all.vertices = TRUE)</pre>
igraph	<pre>igraph::intersection(net, net2, net3)</pre>
network	—

¹ Often, but certainly not always, it will also work if you just use `print(x)`

² These functions have many additional arguments

Graph level indices

Explore the graph

SUMMARIZE THE NETWORK

snafun	snafun::g_summary(x, directed = TRUE)
igraph	–
network	–

COUNT THE NUMBER OF VERTICES

snafun	snafun::count_vertices(x)
igraph	igraph::vcount()
network	network::network.size()

COUNT THE NUMBER OF EDGES

snafun	snafun::count_edges(x)
igraph	igraph::ecount(x) igraph::gsize(x)
network	network::network.edgcount(x)

DENSITY

snafun	snafun::g_density(x, loops = FALSE)
igraph	igraph::edge_density(g)
network	# preferable for bipartite graphs network::network.density(g) # preferable for valued graphs

¹ This function is more general than degree assortativity only: it can be computed on any vertex attribute (via `attrname` or `values`).
² igraph and sna only calculate `Freeman` centralization (snafun does both `Freeman` and `sd`)
³ `\$res` or `\$vector` return the centrality scores

```
sna::gden(g)
```

snafun

```
snafun::g_reciprocity(x)
```

igraph

```
igraph::reciprocity(g)
```

network

```
sna::grecip(g, measure = 'edgewise')
```

snafun

```
snafun::g_transitivity(x)
```

igraph

```
igraph::transitivity(g, type = 'global')
```

network

```
sna::gtrans(g, mode = 'digraph', measure = 'weak', use.adjacency = TRUE)
```

snafun

```
snafun::g_mean_distance(x)
```

igraph

```
igraph::mean_distance(g, directed = TRUE, unconnected = TRUE)
```

network

snafun

```
snafun::g_degree_distribution(x, mode = c("out", "in", "all"),
                             type = c("density", "count"),
                             cumulative = FALSE,
                             loops = FALSE,
                             digits = 3)
```

igraph

```
igraph::degree_distribution(graph, cumulative = FALSE, mode = 'out')
```

³ `'$res'` or `'$vector'` return the centrality scores

network —

```
snafun          snafun::count_dyads(x, echo = TRUE)
```

```
network          sna::dyad.census(g)
```

```
snafun          snafun::count_triads(x, echo = TRUE)
```

```
network          sna::triad.census(g, mode = 'digraph')
```

```
snafun1      snafun::g_assortativity(x, attrname = NULL, values = NULL,
                                vertices = NULL, directed = TRUE, normalized = TRUE)
```

network —

```
snafun      snafun::g_diameter(x, directed = is_directed(x), unconnected = TRUE)
```

```
igraph::diameter(g, directed = TRUE, unconnected = TRUE)
```

```
igraph                                     # what is the vertex pair with the longest geodesic
igraph::farthest_vertices(g)
```

network —

² igraph and sna only calculate 'Freeman' centralization (snafun does both 'Freeman' and 'sd')

file:///C:/Users/20177022/AppData/Local/Temp/Rtmp0MwIOk/fileda7066f43ead.html

Explore the graph

RADIUS

snafun	snafun::g_radius(x, mode = c("all", "out", "in"))
igraph	igraph::radius(graph, mode = c("all", "out", "in", "total"))
network	—

COMPACTNESS

snafun	snafun::g_compactness(x, mode = c("out", "in", "all"))
igraph	—
network	—

CENTRALIZATION

	<pre># a single function to calculate centralization (either `Freeman` or `sd`) # on a wide range of centrality indices</pre>
snafun	<pre>snafun::g_centralize(x, measure = "betweenness", directed = TRUE, mode = c("all", "out", "in"), k = 3, damping = 0.85, normalized = TRUE, method = c("freeman", "sd"))</pre>
igraph ^{2,3}	<pre># general function for Freeman centralization igraph::centralize(scores, theoretical.max = 0, normalized = TRUE) # betweenness centralization igraph::centr_betw(g, directed = TRUE) # closeness centralization igraph::centr_clo(g, mode = 'out', normalized = FALSE) # degree centralization igraph::centr_degree(g, mode = 'all') # eigenvector centralization igraph::centr_eigen(g, directed = TRUE)</pre>
network ²	<pre># general function for Freeman centralization, # include any function that calculates vertex centrality</pre>

¹ This function is more general than degree assortativity only: it can be computed on any vertex attribute (via `attrname` or `values`).

² igraph and sna only calculate `Freeman` centralization (snafun does both `Freeman` and `sd`)

³ `res` or `vector` return the centrality scores

Explore the graph

```
sna::centralization(g, FUN, mode = 'digraph', normalize=TRUE, ...)
```

MIXING MATRIX

snafun	snafun::make_mixingmatrix(x, attrname, by_edge = FALSE, loops = has_loops(x))
igraph	–
network	network::mixingmatrix(object, attrname, useNA = "ifany", expand.bipartite = FALSE)

CORRELATION BETWEEN TWO GRAPHS

snafun	snafun::g_correlation(g1, g2, diag = FALSE)
igraph	–
network	sna::gcor(g1, g2, mode = 'graph')

GRAPH EFFICIENCY

snafun	snafun::g_efficiency(g, diag = FALSE)
igraph	–
network	sna::efficiency(g, diag = FALSE)

SECRECY INDEX

snafun	snafun::g_secretcy(g, type = 0, p = 0.25, digits = 3)
igraph	–
network	–

VULNERABILITY: ATTACK

snafun	snafun::g_vuln_attack(g, mode = c("all", "out", "in"), weight = NA, k = 10, digits = 4)
igraph	–
network	–

¹ This function is more general than degree assortativity only: it can be computed on any vertex attribute (via `attrname` or `values`).

² igraph and sna only calculate `Freeman` centralization (snafun does both `Freeman` and `sd`)

³ `\$res` or `\$vector` return the centrality scores

Explore the graph

VULNERABILITY: EFFICIENCY

snafun	snafun::g_vuln_efficiency(g, method = c("harmonic", "sum"), mode = c("all", "out", "in"), weight = NA, disconnected = c("size", "max", "infinite"), digits = 3)
igraph	—
network	—

VULNERABILITY: PATHS

snafun	snafun::g_vuln_paths(g, mode = c("all", "out", "in"), weight = NULL, digits = 3)
igraph	—
network	—

¹ This function is more general than degree assortativity only: it can be computed on any vertex attribute (via `attrname` or `values`).
² igraph and sna only calculate `Freeman` centralization (snafun does both `Freeman` and `sd`)
³ `\$res` or `\$vector` return the centrality scores

Components and communities

Components and communities	
COUNT THE NUMBER OF COMPONENTS	
snafun	snafun::count_components(x, type = c("weak", "strong"))
igraph	igraph::count_components(graph, mode = c("weak", "strong"))
network	sna::components(dat, connected = "weak")
EXTRACT THE COMPONENTS	
snafun	snafun::extract_components(x, type = c("weak", "strong"))
igraph	igraph::decompose(graph, mode = c("weak", "strong"))
network	–
EXTRACT COMPONENT MEMBERSHIP	
snafun	snafun::extract_component_membership(x, type = c("weak", "strong"))
igraph	igraph::components(graph)\$membership
network	–
EXTRACT CUT VERTICES (ARTICULATION POINTS)	
snafun	snafun::extract_cut_vertices(x, names = TRUE)
igraph	igraph::articulation_points(graph)
network	–
EXTRACT BRIDGES	
snafun	snafun::extract_bridges(x)
igraph	igraph::bridges(graph)
network	–
FAST-GREEDY COMMUNITY DETECTION	

Components and communities	
snafun	<code>snafun::extract_comm_fastgreedy(x, weights = NA, modularity = TRUE, merges = TRUE, membership = TRUE)</code>
igraph	<code>igraph::cluster_fast_greedy(g, merges = TRUE, modularity = TRUE, membership = TRUE, weights = NULL)</code>
network	–
GIRVAN-NEWMAN COMMUNITY DETECTION	
snafun	<code>snafun::extract_comm_girvan(x, weights = NA, directed = TRUE, modularity = TRUE, edge.betweenness = FALSE, bridges = FALSE, merges = TRUE, membership = TRUE)</code>
igraph	<code>igraph::cluster_edge_betweenness(g, weights = NULL, directed = TRUE, edge.betweenness = TRUE, merges = TRUE, bridges = TRUE, modularity = TRUE, membership = TRUE)</code>
network	–
LOUVAIN COMMUNITY DETECTION	
snafun	<code>snafun::extract_comm_louvain(x, weights = NA, resolution = 1)</code>
igraph	<code>igraph::cluster_louvain(graph, weights = NULL, resolution = 1)</code>
network	–
WALKTRAP COMMUNITY DETECTION	
snafun	<code>snafun::extract_comm_walktrap(x, weights = NA, steps = 4, modularity = TRUE, merges = TRUE, membership = TRUE)</code>
igraph	<code>igraph::cluster_walktrap(g, weights = NULL, steps = 4, merges = TRUE, modularity = TRUE, membership = TRUE)</code>
network	–
MERGE COMMUNITY MEMBERSHIP	
snafun	<code>snafun::merge_membership(coms, merges)</code>
igraph	–
network	–
COMMUNITY MEMBERSHIP	

Components and communities	
snafun	snafun::extract_comm_membership(coms)
igraph	igraph::membership(coms)
network	–
NUMBER OF COMMUNITIES	
snafun	snafun::count_communities(coms)
igraph	length(coms)
network	–
COMMUNITY SIZES	
snafun	snafun::extract_comm_sizes(coms)
igraph	igraph::sizes(coms)
network	–
COMMUNITY MODULARITY	
snafun	snafun::extract_comm_modularity(coms)
igraph	igraph::modularity(coms)
network	–
COMMUNITY CROSSING TIES	
snafun	snafun::extract_comm_crossing(coms, graph)
igraph	igraph::crossing(coms, graph)
network	–
EVALUATE COMMUNITIES	
snafun	snafun::evaluate_communities(coms, graph, sil_width = FALSE)

Components and communities

igraph	–
network	–

PLOT SILHOUETTE WIDTHS FOR COMMUNITIES

snafun	evls <- snafun::evaluate_communities(coms, graph) plot(evls)
igraph	–
network	–

PLOT COMMUNITIES

snafun	snafun::plot_communities(coms, graph)
igraph	plot(coms, graph)
network	–

ADD COMMUNITY MEMBERSHIP TO GRAPH

snafun	snafun::add_comm_membership(graph, coms, attr = "community")
igraph	–
network	–

ADD COMMUNITY CROSSING TO GRAPH

snafun	snafun::add_comm_crossing(graph, coms, attr = "crossing")
igraph	–
network	–

SUMMARIZE A COMMUNITY PARTITION

snafun	snafun::summarize_communities(x, graph = NULL)
igraph	–
network	–

PLOT A COMMUNITY DENDROGRAM

snafun	snafun::plot_comm_dendrogram(x, labels = NULL, hang = 0.1)
igraph	igraph::plot_dendrogram(coms)

Components and communities	
network	–
WHICH DETECTION ALGORITHM WAS USED	
snafun	snafun::extract_comm_algorithm(x)
igraph	igraph::algorithm(coms)
network	–
EXTRACT THE COMMUNITY MERGES	
snafun	snafun::extract_comm_merges(x)
igraph	igraph::merges(coms)
network	–

Community results returned by `snafun::extract_comm_*`() are compatible with the wider `igraph` community API. You can inspect memberships, sizes, crossings, modularity, and evaluate the detected communities with `snafun::evaluate_communities()` .

Vertex-level indices

Explore the vertices	
DEGREE	
snafun	snafun::v_degree(x, vids = NULL, mode = c("all", "out", "in"), loops = FALSE, rescaled = FALSE)
igraph	igraph::degree(g, mode = 'out')
network	sna::degree(g, gmode = 'digraph', cmode = 'outdegree')
BETWEENNESS	
snafun	snafun::v_betweenness(x, vids = NULL, directed = TRUE, rescaled = FALSE)
igraph	igraph::betweenness(g, directed = TRUE)
network	sna::betweenness(g, gmode = 'digraph', cmode = 'directed')
FLOW BETWEENNESS	
snafun	snafun::v_flow(x, vids = NULL, mode = c("rawflow", "normflow", "fracflow"), rescaled = FALSE)
igraph	—
network	sna::flowbet(g, gmode = 'digraph', cmode = 'rawflow')
BONACICH POWER CENTRALITY	
snafun	snafun::v_power(x, vids = NULL, exponent = 1, rescaled = FALSE)
igraph	igraph::power_centrality(g)
network	sna::bonpow(g, gmode = 'digraph')
CLOSENESS CENTRALITY	

¹ These bridge measures need a community assignment. `communities` can be the output of `snafun::extract\_comm\_\*()`, a membership vector, or a named list of communities.

² For these shortest-path bridge measures, positive weights are translated to distances via `1/weight`. Zero and negative weights are ignored in the path search; use `weights = NA` for purely topological paths.

Explore the vertices

snafun	snafun::v_closeness(x, vids = NULL, mode = c("all", "out", "in"), rescaled = FALSE)
igraph	igraph::closeness(g, mode = 'all')
network	sna::closeness(g, gmode = 'digraph', cmode = 'directed')

BRIDGE STRENGTH

snafun ¹	snafun::v_bridge_strength(x, communities, use_communities = NULL, type = c("all", "out", "in"), absolute = TRUE, rescaled = FALSE)
igraph	—
network	—

BRIDGE EXPECTED INFLUENCE

snafun ¹	snafun::v_bridge_expected_influence(x, communities, use_communities = NULL, type = c("all", "out", "in"), rescaled = FALSE)
igraph	—
network	—

BRIDGE EXPECTED INFLUENCE (2-STEP)

snafun ¹	snafun::v_bridge_expected_influence2(x, communities, use_communities = NULL, type = c("all", "out", "in"), rescaled = FALSE)
---------------------	--

¹ These bridge measures need a community assignment. `communities` can be the output of `snafun::extract\_comm\_\*()`, a membership vector, or a named list of communities.

² For these shortest-path bridge measures, positive weights are translated to distances via `1/weight`. Zero and negative weights are ignored in the path search; use `weights = NA` for purely topological paths.

Explore the vertices

igraph	–
network	–

BRIDGE CLOSENESS

snafun ²	<pre>snafun::v_bridge_closeness(x, communities, use_communities = NULL, mode = c("out", "in", "all"), weights = NULL, rescaled = FALSE)</pre>
---------------------	---

igraph	–
network	–

BRIDGE BETWEENNESS

snafun ²	<pre>snafun::v_bridge_betweenness(x, communities, use_communities = NULL, directed = NULL, weights = NULL, rescaled = FALSE)</pre>
---------------------	--

igraph	–
network	–

HARMONIC CENTRALITY

snafun	<pre>snafun::v_harmonic(x, vids = NULL, mode = c("all", "out", "in"), rescaled = FALSE)</pre>
--------	---

igraph	<pre>igraph::harmonic centrality(g, vids = V(graph), mode = c("out", "in", "all"), weights = NULL)</pre>
--------	--

network	–
---------	---

STRESS CENTRALITY

snafun	<pre>snafun::v_stress(x, vids = NULL, directed = TRUE, rescaled = FALSE)</pre>
--------	--

igraph	–
--------	---

¹ These bridge measures need a community assignment. `communities` can be the output of `snafun::extract\_comm\_\*()`, a membership vector, or a named list of communities.

² For these shortest-path bridge measures, positive weights are translated to distances via `1/weight`. Zero and negative weights are ignored in the path search; use `weights = NA` for purely topological paths.

Explore the vertices

network	sna::stresscent(g, gmode = 'digraph', cmode = 'directed')
---------	---

ECCENTRICITY

snafun	snafun::v_eccentricity(x, vids = NULL, mode = c("all", "out", "in"), rescaled = FALSE)
--------	--

igraph	igraph::eccentricity(g, mode = 'all')
--------	---------------------------------------

network	—
---------	---

EIGENVECTOR CENTRALITY

snafun	snafun::v_eigenvector(x, directed = TRUE, rescaled = FALSE)
--------	---

igraph	igraph::eigen_centrality(g, directed = TRUE, scale = FALSE)\$vector
--------	---

network	sna::evcent(g, gmode = 'digraph', rescale=FALSE)
---------	--

PAGE RANK

snafun	snafun::v_pagerank(x, vids = NULL, damping = 0.85, directed = TRUE, rescaled = FALSE)
--------	---

igraph	igraph::page_rank(g, vids = V(graph), directed = TRUE, damping = 0.85, weights = NULL)
--------	--

network	—
---------	---

GEO K-PATH

snafun	<pre># without using weights snafun::v_geokpath(x, vids = NULL, mode = c("all", "out", "in"), k = 3, rescaled = FALSE) # if weights are to be used snafun::v_geokpath_w(x, vids = NULL, mode = c("all", "out", "in"), weights = NULL, k = 3)</pre>
--------	---

igraph	—
--------	---

network	—
---------	---

SHAPLEY CENTRALITY

¹ These bridge measures need a community assignment. `communities` can be the output of `snafun::extract\_comm\_\*()`, a membership vector, or a named list of communities.

² For these shortest-path bridge measures, positive weights are translated to distances via `1/weight`. Zero and negative weights are ignored in the path search; use `weights = NA` for purely topological paths.

Explore the vertices

snafun	snafun::v_shapley(x, add.vertex.names = FALSE, vids = NULL, rescaled = FALSE)
igraph	–
network	–

DISTANCES TO AND FROM A VERTEX

snafun	snafun::v_distance(x, mode = c("all", "out", "in"), weights = NULL, count_unnconnected = FALSE)
igraph	igraph::distances(g, v = igraph::V(g), mode = 'out')
network	sna::geodist(g)\$gdist

VERTEX TRANSITIVITY (LOCAL CLUSTERING)

snafun	snafun::v_transitivity(x, vids = NULL, isolates = c("nan", "zero"))
igraph	igraph::transitivity(g, type = 'local')
network	–

BOTTLENECK CENTRALITY

snafun	snafun::v_bottleneck(graph, mode = c("all", "out", "in"), vids = igraph::V(graph), n = 4)
igraph	–
network	–

DIFFUSION CENTRALITY

snafun	snafun::v_diffusion(x, vids = NULL, T = NULL, rescaled = FALSE)
igraph	–
network	–

FRAGMENTATION

snafun	snafun::v_fragment(x, vids = NULL, M = Inf, binary = FALSE,
--------	---

¹ These bridge measures need a community assignment. `communities` can be the output of `snafun::extract\_comm\_\*()`, a membership vector, or a named list of communities.

² For these shortest-path bridge measures, positive weights are translated to distances via `1/weight`. Zero and negative weights are ignored in the path search; use `weights = NA` for purely topological paths.

Explore the vertices

large = TRUE, rescaled = FALSE)

igraph -
network -

VERTEX SECRECY

snafun snafun::v_secrecy(g, type = 1, p = 0.25, digits = 3)

igraph -
network -

WHO ARE THE NEIGHBORS OF A VERTEX

snafun snafun::extract_neighbors(x, vertex, type = c("out", "in", "all"))

igraph igraph::neighbors(g, 'Jane', mode = 'out')
all options
igraph::neighbors(graph, v, mode = c('out', 'in', 'all', 'total'))

network network::get.neighborhood(g, 1, 'out')
all options
network::get.neighborhood(x, v, type = c('out', 'in', 'combined'), na.omit = TRUE)

¹ These bridge measures need a community assignment. `communities` can be the output of `snafun::extract\_comm\_\*()`, a membership vector, or a named list of communities.
² For these shortest-path bridge measures, positive weights are translated to distances via `1/weight`. Zero and negative weights are ignored in the path search; use `weights = NA` for purely topological paths.

Dyad-level indices

Explore the dyads

SHORTEST PATH FOR A GIVEN SET OF VERTICES

snafun	snafun::extract_all_shortest_paths(x, from, to = NULL, mode = c("out", "in", "all"), weights = NULL)
igraph	igraph::all_shortest_paths(g, from = IDofVertex, to = igraph::V(g), mode = 'out')
network	–

¹GEODESIC LENGTHS

snafun	snafun::d_distance(x, mode = c("all", "out", "in"))
igraph	igraph::distances(g, mode = 'out')
network	sna::geodist(g, count.paths = TRUE)\$counts

STRUCTURAL EQUIVALENCE

snafun	snafun::d_structural_equivalence(x, weights = NA, digits = 3, suppressWarnings = TRUE)
igraph	–
network	sna::sedist(g, method = "correlation", mode = "digraph", diag=FALSE)

REGULAR EQUIVALENCE

snafun	snafun::d_regular_equivalence(x, weights = NA, digits = 3)
igraph	–
network	1 - sna::redist(g, method = "catrege", mode = "digraph", diag = FALSE)

EDGE BETWEENNESS

snafun	snafun::d_betweenness(x, directed = TRUE, weights = NA)
igraph	igraph::edge_betweenness(g, directed = FALSE)

¹ The output is a table with an entry per vertex pair

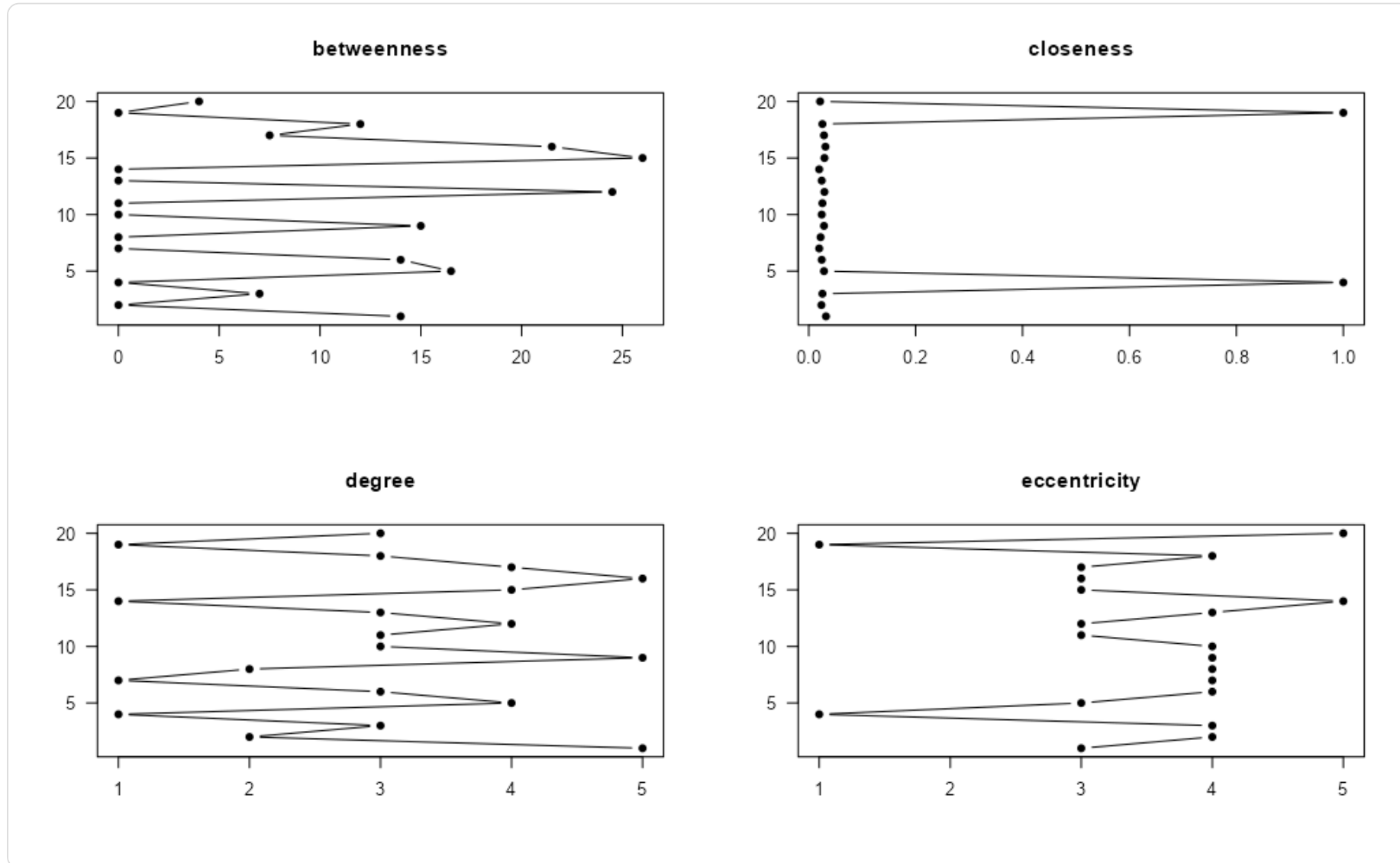
Explore the dyads	
network	–

1 The output is a table with an entry per vertex pair

Plotting

Plotting in snafun

The `snafun` package provides S3 plotting methods, so a plain call to `plot(x)` gives a quick plot regardless of whether `x` is an `igraph` or `network` object. The package also includes a dedicated centrality comparison plot through `snafun::plot_centralities()`.



Example output from `snafun::plot_centralities()`.

Statistical models

Overview table

Statistical network models		
When	Which approach	Function
Dependent vertex attribute explained by a network weight matrix and a matrix of covariates	Network autocorrelation model	snafun::stat_nam
A statistic on a single network	Conditional Uniform Graph test	snafun::stat_cug
Association between two networks	QAP	snafun::stat_qap_cor
A valued dependent network explained by one or more explanatory networks	QAP linear model	snafun::stat_qap_lm
A binary dependent network explained by one or more explanatory networks	QAP logistic model	snafun::stat_qap_logit
A binary or valued dependent network explained by a set of endogenous and exogenous variables	Exponential random graph models	ergm::ergm

Network autocorrelation models

snafun::stat_nam(formula, data, W, model = c("lag", "error", "combined")) — fit a network autocorrelation model, where W is the (row-normalized) network weight matrix.

snafun::stat_nam_summary(x, correlation = TRUE, R2 = TRUE, digits = 3) — a full summary (coefficients, correlations, R²) of a fitted NAM.

snafun::plot_nam(x) — diagnostic plots (e.g. residuals vs. fitted) for a fitted NAM.

Conditional Uniform graphs (CUG)

snafun::stat_cug(x, FUN, cmode = c("size", "edges", "dyad.census"), reps = 1000) — test the statistic FUN on a single network against random graphs conditioned on cmode .

CUG (and QAP below) inference is permutation-based; the table below shows how to randomly permute a network.

Permuting the network	
RANDOMLY PERMUTE THE ORDER OF THE VERTICES	
snafun	—
igraph	igraph::permute(graph = x, permutation = sample(igraph::vcount(x)))
network	sna::rmperm(x)

QAP test

snafun::stat_qap_cor(x, y, controls = NULL, reps = 1000) — test the association between two networks (optionally controlling for others).

QAP linear regression

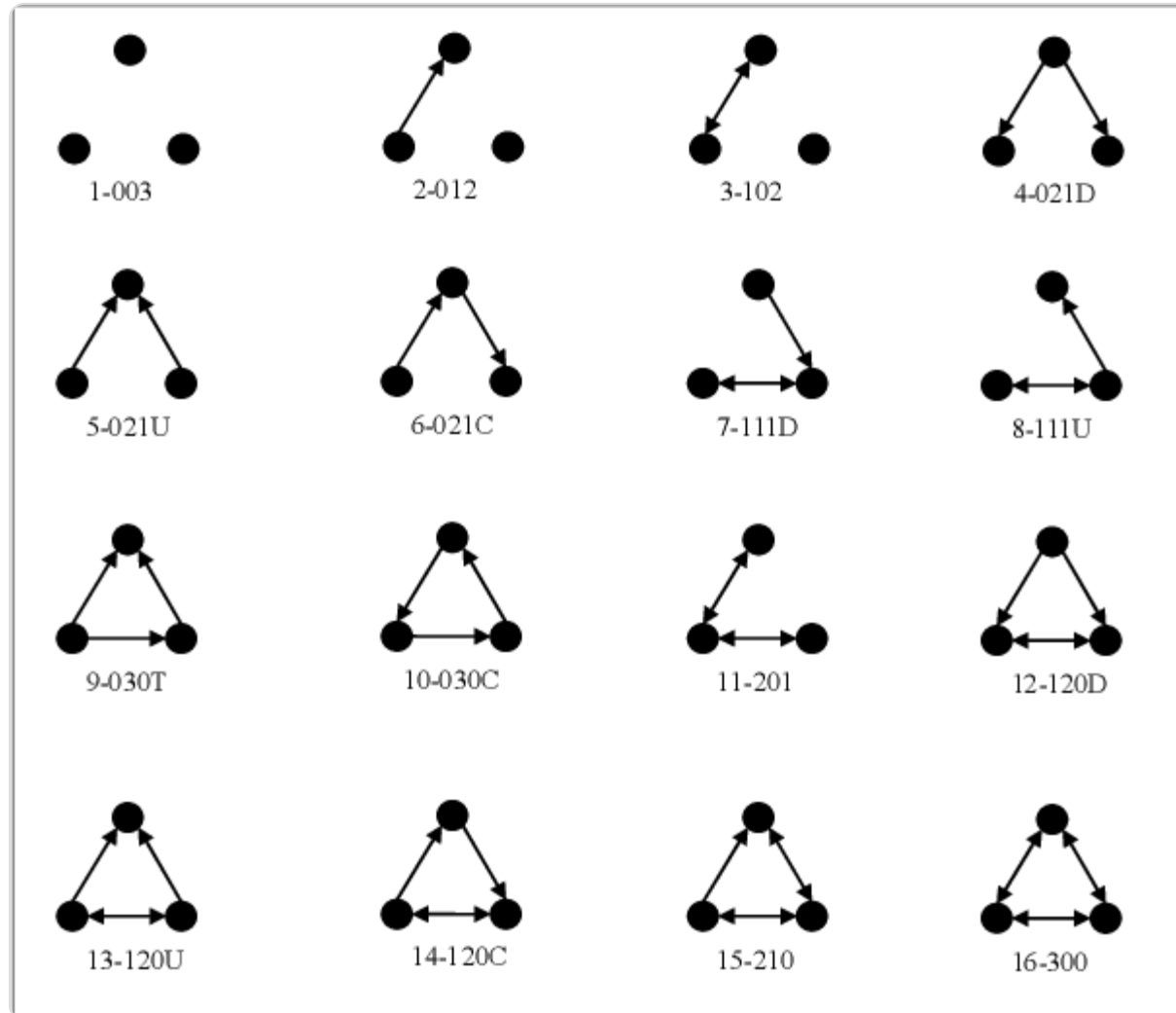
snafun::stat_qap_lm(y, x, reps = 1000) — a valued dependent network y explained by one or more predictor networks x .

QAP logistic regression

`snafun::stat_qap_logit(y, x, reps = 1000)` — same idea, but for a binary dependent network y .

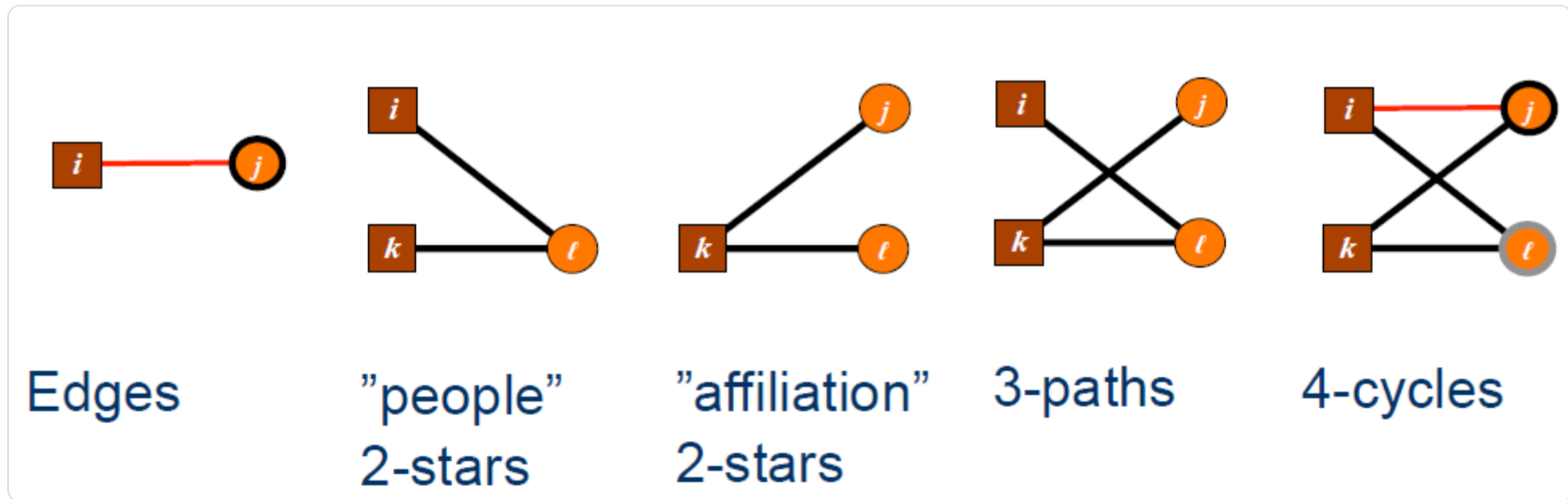
Terms classification for every Exponential Random Graph Model (ERGM)

The static figures below are used in the course to explain common ERGM term families.



Triad census categories used in ERGM teaching.

Bipartite ERGMs



Overview of common bipartite ERGM terms.

ERGM goodness of fit and effect interpretation

`snafun::stat_plot_gof(gof)` — plot an already-computed goodness-of-fit object of a fitted `ergm` or `btergm`.

`snafun::stat_plot_gof_as_btergm(m, silent = FALSE, verbose = TRUE)` — compute AND plot the goodness of fit for a fitted model `m` via the `btergm` engine (works for both `ergm` and `btergm`).

`snafun::stat_ef_int(m, type = "odds")` — translate the effects of a fitted `ergm` into odds ratios (or probabilities).

Temporal networks

Network generation and manipulation

In this course, temporal network work mostly relies on `networkDynamic` , `tsna` , and `ndtv` . The `snafun` API now also supports several active-attribute accessors for `networkDynamic` objects.

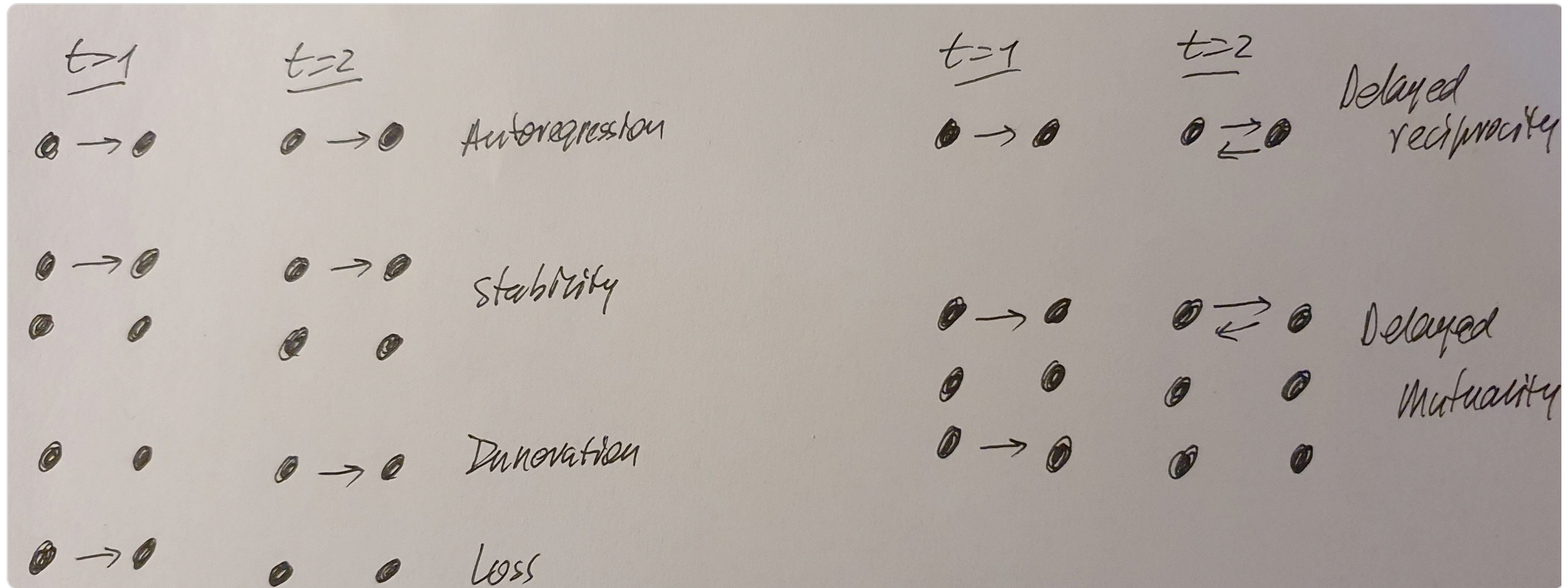
Network measures and descriptives

Temporal descriptives often focus on durations, edge formations, dissolutions, and time-varying ERGM terms. The `snafun` helpers below operate on `networkDynamic` objects.

Temporal network descriptives	
PLOT NETWORK SNAPSHOTS OVER TIME (FILMSTRIP)	
snafun	snafun::plot_network_slices(x, number = 9, start = NULL, end = NULL)
COUNT EDGES PRESENT OVER A TIME INTERVAL	
snafun	snafun::count_edges_in_interval(x, start = NULL, end = NULL, number = 30)
COUNT UNIQUE EDGES OVER A TIME INTERVAL	
snafun	snafun::count_unique_edges_in_interval(x, start = NULL, end = NULL, number = 30, directed = network::is.directed(x))

ERGM for temporal networks

Temporal ERGMs (`btergm`) extend ERGMs with memory, delayed-reciprocity and time-covariate terms. The figure and table below summarize the common temporal terms.



Overview of common temporal ERGM terms.

Temporal effects for the ERGM		
	meaning	btergm
memory		
Positive autoregression	Previous existing edges persist in a next network	memory(type = "autoregression", lag = 1)
Dyadic stability	Both previous existing and non-existing ties are carried over to the current network	memory(type = "stability", lag = 1)
Edge innovation	A non-existing previous tie becomes existent in the current network	memory(type = "innovation", lag = 1)
Edge loss	An existing previous tie is dissolved in the current network	memory(type = "loss", lag = 1)
delayed reciprocity		
reciprocity	if node j is tied to node i at t = 1, does this lead to a reciprocation of that tie back from i to j at t = 2?	delrecip(mutuality = FALSE, lag = 1)
mutuality	if node j is tied to node i at t = 1, does this lead to a reciprocation of that tie back from i to j at t = 2 AND if i is not tied to j at t = 1, will this lead to j not being tied to i at t = 2? This captures a trend away from asymmetry.	delrecip(mutuality = TRUE, lag = 1)
time covariates		
time effect per se	Test for a specific trend (linear or non-linear) for edge formation	timecov(transform = function(t) t)
Time effect of a covariate	Interaction effect to test whether the importance of a covariate increases or decreases over time	timecov(x, transform = function(t) t)
These are ERGM model <i>terms</i> used inside the model formula, e.g. ... ~ edges + memory(type = 'stability') + delrecip(). timecov() is an exported btergm function, while memory() and delrecip() are model terms (documented under btergm's tergm-terms help), so they are not called as btergm::memory().		

Participation shifts

P-shift	Example
Turn receiving	
AB-BA	John talks to Mary, then Mary replies.
AB-B0	John talks to Mary, then Mary addresses the group.
AB-BY	John talks to Mary, then Mary talks to Irene.
Turn claiming	
A0-X0	John talks to the group, then Frank talks to the group.
A0-XA	John talks to the group, then Frank talks to John.
A0-XY	John talks to the group, then Frank talks to Mary.
Turn usurping	
AB-X0	John talks to Mary, then Frank talks to the group.
AB-XA	John talks to Mary, then Frank talks to John.
AB-XB	John talks to Mary, then Frank addresses Mary.
AB-XY	John talks to Mary, then Frank addresses Irene.
Turn continuing	
A0-AY	John talks to the group, then addresses Mary.
AB-A0	John talks to Mary, then makes a remark to the group.
AB-AY	John talks to Mary then to Irene.
<i>Note:</i> The initial speaker is always labeled A, and the initial target B, unless the group is addressed (or the target was ambiguous), in which case the target is 0. Then the shift is summarized in the form [speaker₁][target₁]-[speaker₂][target₂], with A or B appearing after the hyphen only if the initial speaker or target serves in one of these two positions after the shift. When the speaker after the shift is someone other than A or B, X is used, and when the target after the shift is someone other than A, B, or the group, Y is used.	

Participation-shift categories used in the temporal networks part of the course.